Application-Time Periods

Application-time period tables define a time period between two temporal columns, for application-level data versioning.

Extending system-versioned tables, MariaDB supports application-time period tables. Time periods are defined by a range between two temporal columns. The columns must be of the same temporal data type, i.e. DATE, TIMESTAMP or DATETIME (TIME and YEAR are not supported), and of the same width.

Using time periods implicitly defines the two columns as `NOT NULL`. It also adds a constraint to check whether the first value is less than the second value. The constraint is invisible to SHOW CREATE TABLE statements. The name of this constraint is prefixed by the time period name, to avoid conflict with other constraints.

Creating Tables with Time Periods

To create a table with a time period, use a CREATE TABLE statement with the `PERIOD` table option.

```
CREATE TABLE t1(
   name VARCHAR(50),
   date_1 DATE,
   date_2 DATE,
   PERIOD FOR date_period(date_1, date_2));
```

This creates a table with a `time_period` period and populates the table with

some basic temporal values.

Examples are available in the MariaDB Server source code, at `mysql-test/suite/period/r/create.result`.

Adding and Removing Time Periods

The ALTER TABLE statement now supports syntax for adding and removing time periods from a table. To add a period, use the `ADD PERIOD` clause:

```
CREATE OR REPLACE TABLE rooms (
 room_number INT,
 guest_name VARCHAR(255),
 checkin DATE,
 checkout DATE
 );

ALTER TABLE rooms ADD PERIOD FOR p(checkin,checkout);
```

To remove a period, use the `DROP PERIOD` clause:

```
ALTER TABLE rooms DROP PERIOD FOR p;
```

Both `ADD PERIOD` and `DROP PERIOD` clauses include an option to handle whether the period already exists:

```
ALTER TABLE rooms ADD PERIOD IF NOT EXISTS FOR p(checkin,checkout);

ALTER TABLE rooms DROP PERIOD IF EXISTS FOR p;
```

Deletion by Portion

You can also remove rows that fall within certain time periods.

When MariaDB executes a `DELETE FOR PORTION` statement, it removes the row:

When the row period falls completely within the delete period, it removes the row.

When the row period overlaps the delete period, it shrinks the row, removing the overlap from the first or second row period value.

When the delete period falls completely within the row period, it splits the row into two rows. The first row runs from the starting row period to the starting delete period. The second runs from the ending delete period to the ending row period.

To test this, first populate the table with some data to operate on:

```
CREATE TABLE t1(
   name VARCHAR(50),
   date_1 DATE,
   date_2 DATE,
   PERIOD FOR date_period(date_1, date_2));

INSERT INTO t1 (name, date_1, date_2) VALUES
    ('a', '1999-01-01', '2000-01-01'),
    ('b', '1999-01-01', '2018-12-12'),
    ('c', '1999-01-01', '2017-01-01'),
    ('d', '2017-01-01', '2019-01-01');

SELECT * FROM t1;
```

```
+------+------------+------------+
| name | date_1     | date_2     |
+------+------------+------------+
| a    | 1999-01-01 | 2000-01-01 |
| b    | 1999-01-01 | 2018-12-12 |
| c    | 1999-01-01 | 2017-01-01 |
| d    | 2017-01-01 | 2019-01-01 |
+------+------------+------------+
```

Then, run the `DELETE FOR PORTION` statement:

```
DELETE FROM t1
FOR PORTION OF date_period
    FROM '2001-01-01' TO '2018-01-01';
Query OK, 3 rows affected (0.028 sec)

SELECT * FROM t1 ORDER BY name;
+------+------------+------------+
| name | date_1     | date_2     |
+------+------------+------------+
| a    | 1999-01-01 | 2000-01-01 |
| b    | 1999-01-01 | 2001-01-01 |
| b    | 2018-01-01 | 2018-12-12 |
| c    | 1999-01-01 | 2001-01-01 |
| d    | 2018-01-01 | 2019-01-01 |
+------+------------+------------+
```

Here:

`a` is unchanged, as the range falls entirely out of the specified portion to be deleted.

`b`, with values ranging from 1999 to 2018, is split into two rows, 1999 to 2000

and 2018-01 to 2018-12 (i.e. one extra row has been inserted).

`c`, with values ranging from 1999 to 2017, where only the upper value falls within the portion to be deleted, has been shrunk to 1999 to 2001.

`d`, with values ranging from 2017 to 2019, where only the lower value falls within the portion to be deleted, has been shrunk to 2018 to 2019.

The `DELETE FOR PORTION` statement has the following restrictions

The `FROM...TO` clause must be constant.

Multi-delete is not supported.

If there are `DELETE` or `INSERT` triggers, any matched row is deleted, and then one or two rows are inserted. If the record is deleted completely, nothing is inserted.

Updating by Portion

The UPDATE syntax now supports `UPDATE FOR PORTION`, which modifies rows based on their occurrence in a range:

To test it, first populate the table with some data:

```
TRUNCATE t1;

INSERT INTO t1 (name, date_1, date_2) VALUES
    ('a', '1999-01-01', '2000-01-01'),
    ('b', '1999-01-01', '2018-12-12'),
    ('c', '1999-01-01', '2017-01-01'),
    ('d', '2017-01-01', '2019-01-01');
```

```
SELECT * FROM t1;
+------+------------+------------+
| name | date_1     | date_2     |
+------+------------+------------+
| a    | 1999-01-01 | 2000-01-01 |
| b    | 1999-01-01 | 2018-12-12 |
| c    | 1999-01-01 | 2017-01-01 |
| d    | 2017-01-01 | 2019-01-01 |
+------+------------+------------+
```

Then run the update:

```
UPDATE t1 FOR PORTION OF date_period
  FROM '2000-01-01' TO '2018-01-01'
SET name = CONCAT(name,'_original');

SELECT * FROM t1 ORDER BY name;
+------------+------------+------------+
| name       | date_1     | date_2     |
+------------+------------+------------+
| a          | 1999-01-01 | 2000-01-01 |
| b          | 1999-01-01 | 2000-01-01 |
| b          | 2018-01-01 | 2018-12-12 |
| b_original | 2000-01-01 | 2018-01-01 |
| c          | 1999-01-01 | 2000-01-01 |
| c_original | 2000-01-01 | 2017-01-01 |
| d          | 2018-01-01 | 2019-01-01 |
| d_original | 2017-01-01 | 2018-01-01 |
+------------+------------+------------+
```

a is unchanged, as the range falls entirely out of the specified portion to be updated.

For `b`, with years ranging from 1999 to 2018, two extra rows are inserted, with ranges 1999-01 to 2000-01 and 2018-01 to 2018-12. The original row's period has been shrunk to years 2000 and 2018, and the `name` field has got "_original" appended.

`c`, with values ranging from 1999 to 2017, where only the upper value falls within the portion to be updated, has been shrunk to 1999 to 2001.

`d`, with values ranging from 2017 to 2019, where only the lower value falls within the portion to be updated, has been shrunk to 2018 to 2019.

Original rows affected by the update have "_original" appended to the name.

The `UPDATE FOR PORTION` statement has the following limitations:

The operation cannot modify the two temporal columns used by the time period.

The operation cannot reference period values in the `SET` expression.

`FROM...TO` expressions must be constant.

WITHOUT OVERLAPS

This clause is available from MariaDB 10.5.3.

`WITHOUT OVERLAPS` allows to create an index specifying that application time periods should not overlap. An index constrained by `WITHOUT OVERLAPS` is required to be either a primary key or a unique index.

Take the following example, an application time period table for a booking system:

```
CREATE OR REPLACE TABLE rooms (
 room_number INT,
 guest_name VARCHAR(255),
 checkin DATE,
 checkout DATE,
 PERIOD FOR p(checkin,checkout)
 );

INSERT INTO rooms VALUES
 (1, 'Regina', '2020-10-01', '2020-10-03'),
 (2, 'Cochise', '2020-10-02', '2020-10-05'),
 (1, 'Nowell', '2020-10-03', '2020-10-07'),
 (2, 'Eusebius', '2020-10-04', '2020-10-06');
```

Our system is not intended to permit overlapping bookings, so the fourth record above should not have been inserted. Using `WITHOUT OVERLAPS` in a unique index (in this case based on a combination of room number and the application time period) allows to specify this constraint in the table definition.

```
CREATE OR REPLACE TABLE rooms (
 room_number INT,
 guest_name VARCHAR(255),
 checkin DATE,
 checkout DATE,
 PERIOD FOR p(checkin,checkout),
 UNIQUE (room_number, p WITHOUT OVERLAPS)
 );

INSERT INTO rooms VALUES
 (1, 'Regina', '2020-10-01', '2020-10-03'),
 (2, 'Cochise', '2020-10-02', '2020-10-05'),
 (1, 'Nowell', '2020-10-03', '2020-10-07'),
```

```
 (2, 'Eusebius', '2020-10-04', '2020-10-06');
ERROR 1062 (23000): Duplicate entry '2-2020-10-06-2020-10-04' for key 'room_numb
```

Information Schema

Information Schema support for application time period tables is available from MariaDB 11.4.

Information Schema contains the following support for application time period tables:

Further Examples

The implicit change from `NULL` to `NOT NULL`:

```
CREATE TABLE `t2` (
  `id` int(11) DEFAULT NULL,
  `d1` datetime DEFAULT NULL,
  `d2` datetime DEFAULT NULL
) ENGINE=InnoDB DEFAULT CHARSET=latin1;

ALTER TABLE t2 ADD PERIOD FOR p(d1,d2);

SHOW CREATE TABLE t2\G
*************************** 1. row ***************************
       Table: t2
Create Table: CREATE TABLE `t2` (
* //a// is *unchanged*, as the range falls entirely out of the specified portion
* For //b//, with years ranging from 1999 to 2018, two extra rows are *inserted*
* //c//, with values ranging from 1999 to 2017, where only the upper value falls
* //d//, with values ranging from 2017 to 2019, where only the lower value falls
* Original rows affected by the update have "_original" appended to the ##name##
  `id` int(11) DEFAULT NULL,
```

```
  `d1` datetime NOT NULL,
  `d2` datetime NOT NULL,
  PERIOD FOR `p` (`d1`, `d2`)
) ENGINE=InnoDB DEFAULT CHARSET=latin1
```

Due to this constraint, trying to add a time period where `NULL` data already exist fails:

```
CREATE OR REPLACE TABLE `t2` (
  `id` int(11) DEFAULT NULL,
  `d1` datetime DEFAULT NULL,
  `d2` datetime DEFAULT NULL
) ENGINE=InnoDB DEFAULT CHARSET=latin1;

INSERT INTO t2(id) VALUES(1);

ALTER TABLE t2 ADD PERIOD FOR p(d1,d2);
ERROR 1265 (01000): Data truncated for column 'd1' at row 1
```

See Also

This page is licensed: CC BY-SA / Gnu FDL

System-Versioned Tables

MariaDB supports temporal data tables in the form of system-versioning tables (allowing you to query and operate on historic data, discussed below), (allow you to query and operate on a temporal range of data), and (which combine both system-versioning and).

System-versioned tables store the history of all changes, not only data which is currently applicable. This allows data analysis for any point in time, auditing of changes and comparison of data from different points in time. Typical uses cases are:

Forensic analysis & legal requirements to store data for N years.

Data analytics (retrospective, trends etc.), e.g. to get your staff information as of one year ago.

Point-in-time recovery - recover a table state as of particular point in time.

System-versioned tables were first introduced in the SQL:2011 standard.

The keyword `SYSTEM` is used in the `SYSTEM VERSIONING` clause. Using `SYSTEM` as an unquoted column name in an `ALTER TABLE` is not permitted by some parser rules. Even though `SYSTEM` is not a fully reserved word in Oracle mode, the `ADD` statement will result in a syntax error.

Always provide `SYSTEM` when using it as a column name: `system`. See the Reserved Words page for additional examples and a detailed explanation.

Creating a System-Versioned Table

The CREATE TABLE syntax has been extended to permit creating a system-versioned table. To be system-versioned, according to SQL:2011, a table must have two generated columns, a period, and a special table option clause:

```
CREATE TABLE t(
   x INT,
   start_timestamp TIMESTAMP(6) GENERATED ALWAYS AS ROW START,
   end_timestamp TIMESTAMP(6) GENERATED ALWAYS AS ROW END,
   PERIOD FOR SYSTEM_TIME(start_timestamp, end_timestamp)
) WITH SYSTEM VERSIONING;
```

In MariaDB, you can use a simplified syntax:

```
CREATE TABLE t (
   x INT
) WITH SYSTEM VERSIONING;
```

In the latter case, no extra columns will be created, and they won't clutter the output of, say, `SELECT * FROM t`. The versioning information will still be stored, and it can be accessed via the pseudo-columns `ROW_START` and `ROW_END`:

```
SELECT x, ROW_START, ROW_END FROM t;
```

Adding or Removing System Versioning To/From a Table

An existing table can be altered to enable system versioning for it.

```
CREATE TABLE t(
  x INT
);

ALTER TABLE t ADD SYSTEM VERSIONING;

SHOW CREATE TABLE t\G
*************************** 1. row ***************************
       Table: t
Create Table: CREATE TABLE `t` (
  `x` int(11) DEFAULT NULL
) ENGINE=InnoDB DEFAULT CHARSET=latin1 WITH SYSTEM VERSIONING
```

Similarly, system versioning can be removed from a table:

```
ALTER TABLE t DROP SYSTEM VERSIONING;

SHOW CREATE TABLE t\G
*************************** 1. row ***************************
       Table: t
Create Table: CREATE TABLE `t` (
  `x` int(11) DEFAULT NULL
) ENGINE=InnoDB DEFAULT CHARSET=latin1
```

You can also add system versioning with all columns created explicitly:

```
ALTER TABLE t ADD COLUMN ts TIMESTAMP(6) GENERATED ALWAYS AS ROW START,
              ADD COLUMN te TIMESTAMP(6) GENERATED ALWAYS AS ROW END,
              ADD PERIOD FOR SYSTEM_TIME(ts, te),
              ADD SYSTEM VERSIONING;
```

```
SHOW CREATE TABLE t\G
*************************** 1. row ***************************
       Table: t
Create Table: CREATE TABLE `t` (
  `x` int(11) DEFAULT NULL,
  `ts` timestamp(6) GENERATED ALWAYS AS ROW START,
  `te` timestamp(6) GENERATED ALWAYS AS ROW END,
  PERIOD FOR SYSTEM_TIME (`ts`, `te`)
) ENGINE=InnoDB DEFAULT CHARSET=latin1 WITH SYSTEM VERSIONING
```

It is possible to convert a versioned table from implicit to explicit `row_start`/`row_end` columns. Note that, in order to do any `ALTER` on a system versioned table, system_versioning_alter_history must be set to `KEEP`.

```
CREATE OR REPLACE TABLE t1 (x INT) WITH SYSTEM VERSIONING;

SET system_versioning_alter_history = keep;

ALTER TABLE t1 ADD COLUMN rs TIMESTAMP(6) AS ROW START,
  ADD COLUMN re TIMESTAMP(6) AS ROW END, ADD PERIOD FOR SYSTEM_TIME (rs,re)
```

Inserting Data

When data is inserted into a system-versioned table, it is given a *row_start* value of the current timestamp, and a *row_end* value of FROM_UNIXTIME (`2147483647.999999`). The current timestamp can be adjusted by setting the timestamp system variable:

```
SELECT NOW();
+---------------------+
| NOW()               |
```

```
+---------------------+
| 2022-10-24 23:09:38 |
+---------------------+

INSERT INTO t VALUES(1);

SET @@timestamp = UNIX_TIMESTAMP('2033-10-24');

INSERT INTO t VALUES(2);

SET @@timestamp = default;

INSERT INTO t VALUES(3);

SELECT a,row_start,row_end FROM t;
+------+----------------------------+----------------------------+
| a    | row_start                  | row_end                    |
+------+----------------------------+----------------------------+
|    1 | 2022-10-24 23:09:38.951347 | 2038-01-19 05:14:07.999999 |
|    2 | 2033-10-24 00:00:00.000000 | 2038-01-19 05:14:07.999999 |
|    3 | 2022-10-24 23:09:38.961857 | 2038-01-19 05:14:07.999999 |
+------+----------------------------+----------------------------+
```

Updating Data

When an `UPDATE` statement modifies a row in a system-versioned table, the server writes a new history row capturing the previous state of the row.

A history row is created for every `UPDATE` statement that includes a versioned column in its `SET` list, regardless of whether any column value actually changes. Running `UPDATE t SET x = x` against an existing row of a versioned table still produces a new history row — what determines history-row creation is the presence of a versioned column in the `SET` list, not

whether the supplied values differ from the stored values. `UPDATE` statements whose `SET` list references only columns declared `WITHOUT SYSTEM VERSIONING` do not produce a history row.

```
CREATE OR REPLACE TABLE t (x INT) WITH SYSTEM VERSIONING;
INSERT INTO t VALUES (1);

-- This UPDATE changes nothing about the stored value of x,
-- but still produces a new history row:
UPDATE t SET x = 1;

SELECT x, ROW_START, ROW_END FROM t FOR SYSTEM_TIME ALL;
```

This statement-level behavior applies to the default timestamp-based versioning. For transaction-precise versioning, history generation is handled at the storage-engine level by InnoDB's MVCC and is tied to actual row-level changes.

Reasoning for the behavior: Logging updates that do not change data values ensures a complete audit trail and maintains consistent server behavior. In MariaDB, an `UPDATE` statement executes its associated triggers regardless of whether the data values actually change. Recording a historical row for these statements ensures that system-versioned tables follow this standard query logic. Furthermore, capturing the exact timestamp of these updates provides vital context for auditing and troubleshooting. It proves that an application explicitly targeted and processed a specific record at that exact moment—valuable information that would be lost if the server silently ignored the event.

See MDEV-23446 for related discussion.

Querying Historical Data

SELECT

To query the historical data one uses the clause `FOR SYSTEM_TIME` directly after the table name (before the table alias, if any). SQL:2011 provides three syntactic extensions:

`AS OF` is used to see the table as it was at a specific point in time in the past:

```
SELECT * FROM t FOR SYSTEM_TIME AS OF TIMESTAMP'2016-10-09 08:07:06';
```

`BETWEEN start AND end` will show all rows that were visible at any point between two specified points in time. It works inclusively, a row visible exactly at start or exactly at end will be shown too.

```
SELECT * FROM t FOR SYSTEM_TIME BETWEEN (NOW() - INTERVAL 1 YEAR) AND NOW();
```

`FROM start TO end` will also show all rows that were visible at any point between two specified points in time, including start, but excluding end.

```
SELECT * FROM t FOR SYSTEM_TIME FROM '2016-01-01 00:00:00' TO '2017-01-01 00:00:
```

Additionally, MariaDB implements a non-standard extension.

`ALL` will show all rows, historical and current.

```
SELECT * FROM t FOR SYSTEM_TIME ALL;
```

If the `FOR SYSTEM_TIME` clause is not used, the table shows the *current* data. This is usually the same as if you had specified `FOR SYSTEM_TIME AS OF CURRENT_TIMESTAMP`, unless you've adjusted the *row_start* value:

```
CREATE OR REPLACE TABLE t (a int) WITH SYSTEM VERSIONING;

SELECT NOW();
+---------------------+
| NOW()               |
+---------------------+
| 2022-10-24 23:43:37 |
+---------------------+

INSERT INTO t VALUES (1);

SET @@timestamp = UNIX_TIMESTAMP('2033-03-03');

INSERT INTO t VALUES (2);

DELETE FROM t;

SET @@timestamp = default;


SELECT a, row_start, row_end FROM t FOR SYSTEM_TIME ALL;
+------+----------------------------+----------------------------+
| a    | row_start                  | row_end                    |
+------+----------------------------+----------------------------+
|    1 | 2022-10-24 23:43:37.192725 | 2033-03-03 00:00:00.000000 |
|    2 | 2033-03-03 00:00:00.000000 | 2033-03-03 00:00:00.000000 |
+------+----------------------------+----------------------------+
2 rows in set (0.000 sec)
```

```
SELECT a, row_start, row_end FROM t FOR SYSTEM_TIME AS OF CURRENT_TIMESTAMP;
+------+----------------------------+----------------------------+
| a    | row_start                  | row_end                    |
+------+----------------------------+----------------------------+
|    1 | 2022-10-24 23:43:37.192725 | 2033-03-03 00:00:00.000000 |
+------+----------------------------+----------------------------+
1 row in set (0.000 sec)


SELECT a, row_start, row_end FROM t;
Empty set (0.001 sec)
```

Views and Subqueries

When a system-versioned table is used in a view or in a subquery in the from clause, `FOR SYSTEM_TIME` can be used directly in the view or subquery body, or (non-standard) applied to the whole view when it's being used in a `SELECT`:

```
CREATE VIEW v1 AS SELECT * FROM t FOR SYSTEM_TIME AS OF TIMESTAMP'2016-10-09 08:
```

Or:

```
CREATE VIEW v1 AS SELECT * FROM t;
SELECT * FROM v1 FOR SYSTEM_TIME AS OF TIMESTAMP'2016-10-09 08:07:06';
```

Use in Replication and Binary Logs

Tables that use system-versioning implicitly add the `row_end` column to the Primary Key. While this is generally not an issue for most use cases, it can lead to problems when re-applying write statements from the binary log or in

replication environments, where a primary retries an SQL statement on the replica.

Specifically, these writes include a value on the `row_end` column containing the timestamp from when the write was initially made. The re-occurrence of the Primary Key with the old system-versioning columns raises an error due to the duplication.

To mitigate this with MariaDB Replication, set the secure_timestamp system variable to `YES` on the replica. When set, the replica uses its own system clock when applying to the row log, meaning that the primary can retry as many times as needed without causing a conflict. The retries generate new historical rows with new values for the `row_start` and `row_end` columns.

Transaction-Precise History in InnoDB

A point in time when a row was inserted or deleted does not necessarily mean that a change became visible at the same moment. With transactional tables, a row might have been inserted in a long transaction, and became visible hours after it was inserted.

For some applications — for example, when doing data analytics on one-year-old data — this distinction does not matter much. For others — forensic analysis — it might be crucial.

MariaDB supports transaction-precise history (only for the InnoDB storage engine) that allows seeing the data exactly as it would've been seen by a new connection doing a `SELECT` at the specified point in time — rows inserted *before* that point, but committed *after* will not be shown.

To use transaction-precise history, InnoDB needs to remember not timestamps, but transaction identifier per row. This is done by creating

generated columns as `BIGINT UNSIGNED`, not `TIMESTAMP(6)`:

```
CREATE TABLE t(
   x INT,
   start_trxid BIGINT UNSIGNED GENERATED ALWAYS AS ROW START,
   end_trxid BIGINT UNSIGNED GENERATED ALWAYS AS ROW END,
   PERIOD FOR SYSTEM_TIME(start_trxid, end_trxid)
) WITH SYSTEM VERSIONING;
```

These columns must be specified explicitly, but they can be made INVISIBLE to avoid cluttering `SELECT *` output.

Note that if you are using an engine that does not support system versioning with transaction ids, you will get an error like "`start_trxid` must be of type `TIMESTAMP(6)` for system-versioned table `t`".

When one uses transaction-precise history, one can optionally use transaction identifiers in the `FOR SYSTEM_TIME` clause:

```
SELECT * FROM t FOR SYSTEM_TIME AS OF TRANSACTION 12345;
```

This shows the data, exactly as it was seen by the transaction with the identifier 12345.

Data for this feature is stored in the mysql.transaction_registry table.

Limitations

`PARTITION BY SYSTEM_TIME` is not supported when using transaction-precise system versioning. Transaction-precise history tracks row versions using transaction identifiers instead of timestamps, so, `SYSTEM_TIME` cannot be

used to partition transaction-precise system history. If partitioning is required, use the default timestamp-based versioning.

Storing the History Separately

When the history is stored together with the current data, it increases the size of the table, so current data queries — table scans and index searches — will take more time, because they will need to skip over historical data. If most queries on that table use only current data, it might make sense to store the history separately, to reduce the overhead from versioning.

This is done by partitioning the table by `SYSTEM_TIME`. Because of the partition pruning optimization, all current data queries will only access one partition, the one that stores current data.

This example shows how to create such a partitioned table:

```
CREATE TABLE t (x INT) WITH SYSTEM VERSIONING
  PARTITION BY SYSTEM_TIME (
    PARTITION p_hist HISTORY,
    PARTITION p_cur CURRENT
  );
```

In this example, all history will be stored in the partition `p_hist` while all current data will be in the partition `p_cur`. The table must have exactly one current partition and at least one historical partition.

Partitioning by `SYSTEM_TIME` also supports automatic partition rotation. You can rotate historical partitions by time or by size. This example shows how to rotate partitions by size:

```
CREATE TABLE t (x INT) WITH SYSTEM VERSIONING
  PARTITION BY SYSTEM_TIME LIMIT 100000 (
    PARTITION p0 HISTORY,
    PARTITION p1 HISTORY,
    PARTITION pcur CURRENT
  );
```

MariaDB starts writing history rows into partition `p0`, and at the end of the statement that wrote the 100000th row, MariaDB will switch to partition `p1`. There are only two historical partitions, so when `p1` overflows, MariaDB will issue a warning, but will continue writing into it.

Note on `LIMIT` precision: When using `LIMIT` to specify partition rotation, the specified value is just an estimate of the partition size. Although the partition's intended size is specified by the `LIMIT` setting, the actual number of rows may differ. This occurs because partition rotation is triggered during internal operations rather than when the specified row count is met. Partitions may therefore contain significantly more or fewer rows than the set `LIMIT` value.

Similarly, one can rotate partitions by time:

```
CREATE TABLE t (x INT) WITH SYSTEM VERSIONING
  PARTITION BY SYSTEM_TIME INTERVAL 1 WEEK (
    PARTITION p0 HISTORY,
    PARTITION p1 HISTORY,
    PARTITION p2 HISTORY,
    PARTITION pcur CURRENT
  );
```

This means that the history for the first week after the table was created will

be stored in `p0`. The history for the second week — in `p1`, and all later history will go into `p2`. One can see the exact rotation time for each partition in the INFORMATION_SCHEMA.PARTITIONS table.

It is possible to combine partitioning by `SYSTEM_TIME` and subpartitions:

```
CREATE TABLE t (x INT) WITH SYSTEM VERSIONING
  PARTITION BY SYSTEM_TIME
    SUBPARTITION BY KEY (x)
    SUBPARTITIONS 4 (
    PARTITION ph HISTORY,
    PARTITION pc CURRENT
  );
```

Default Partitions

Default Partitions are available from MariaDB 10.5.

Since partitioning by current and historical data is such a typical use case, it is possible to use a simplified statement to do so. Instead of the following statement:

```
CREATE TABLE t (x INT) WITH SYSTEM VERSIONING
  PARTITION BY SYSTEM_TIME (
    PARTITION p0 HISTORY,
    PARTITION pn CURRENT
);
```

You can use:

```
CREATE TABLE t (x INT) WITH SYSTEM VERSIONING
```

```
  PARTITION BY SYSTEM_TIME;
```

You can also specify the number of partitions, which is useful if you want to rotate history by time, for example:

```
CREATE TABLE t (x INT) WITH SYSTEM VERSIONING
  PARTITION BY SYSTEM_TIME
    INTERVAL 1 MONTH
    PARTITIONS 12;
```

Specifying the number of partitions without specifying a rotation condition results in a warning:

```
CREATE OR REPLACE TABLE t (x INT) WITH SYSTEM VERSIONING
  PARTITION BY SYSTEM_TIME PARTITIONS 12;
Query OK, 0 rows affected, 1 warning (0.518 sec)

Warning (Code 4115): Maybe missing parameters: no rotation condition for multipl
```

While specifying only one partition results in an error:

```
CREATE OR REPLACE TABLE t (x INT) WITH SYSTEM VERSIONING
  PARTITION BY SYSTEM_TIME PARTITIONS 1;
ERROR 4128 (HY000): Wrong partitions for `t`: must have at least one HISTORY and
```

Automatically Creating Partitions

Creating partitions with `AUTO` is available from MariaDB 10.9.1.

The `AUTO` keyword can be used to automatically create history partitions:

```
CREATE TABLE t1 (x INT) WITH SYSTEM VERSIONING
    PARTITION BY SYSTEM_TIME INTERVAL 1 HOUR AUTO;

CREATE TABLE t1 (x INT) WITH SYSTEM VERSIONING
   PARTITION BY SYSTEM_TIME INTERVAL 1 MONTH
   STARTS '2021-01-01 00:00:00' AUTO PARTITIONS 12;

CREATE TABLE t1 (x INT) WITH SYSTEM VERSIONING
  PARTITION BY SYSTEM_TIME LIMIT 1000 AUTO;
```

Or with explicit partitions:

```
CREATE TABLE t1 (x INT) WITH SYSTEM VERSIONING
  PARTITION BY SYSTEM_TIME INTERVAL 1 HOUR AUTO
  (PARTITION p0 HISTORY, PARTITION pn CURRENT);
```

To disable or enable auto-creation, one can use `ALTER TABLE` by adding or removing `AUTO` from the partitioning specification:

```
CREATE TABLE t1 (x INT) WITH SYSTEM VERSIONING
  PARTITION BY SYSTEM_TIME INTERVAL 1 HOUR AUTO;

# Disables auto-creation:
ALTER TABLE t1 PARTITION BY SYSTEM_TIME INTERVAL 1 HOUR;

# Enables auto-creation:
ALTER TABLE t1 PARTITION BY SYSTEM_TIME INTERVAL 1 HOUR AUTO;
```

If the rest of the partitioning specification is identical to `CREATE TABLE`, no repartitioning will be done (for details see MDEV-27328).

> Partitioning by SYSTEM_TIME is not available for tables that use

> transaction-precise history. If partitioning is required, use the default timestamp-based system versioning instead.

Removing Old History

Because it stores all the history, a system-versioned table might grow very large over time. There are many options to trim down the space and remove the old history.

One can completely drop the versioning from the table and add it back again, this deletes all the history:

```
ALTER TABLE t DROP SYSTEM VERSIONING;
ALTER TABLE t ADD SYSTEM VERSIONING;
```

It might be a rather time-consuming operation, though, as the table needs to be rebuilt, possibly twice (depending on the storage engine).

Another option would be to use partitioning and drop some of historical partitions:

```
ALTER TABLE t DROP PARTITION p0;
```

You cannot drop a current partition or the only historical partition.

As a third option; you can use a variant of the DELETE statement to prune the history:

Or only old history up to a specific point in time:

```
DELETE HISTORY FROM t BEFORE SYSTEM_TIME '2016-10-09 08:07:06';
```

Or to a specific transaction (with `BEFORE SYSTEM_TIME TRANSACTION xxx`).

To protect the integrity of the history, this statement requires a special DELETE HISTORY privilege.

Currently, using the `DELETE HISTORY` statement with a `BEFORE SYSTEM_TIME` greater than the `ROW_END` of the active records (as a TIMESTAMP, this has a maximum value of `'2038-01-19 03:14:07'` UTC) results in the historical records being dropped, and the active records being deleted and moved to history. See MDEV-25468.

The TRUNCATE TABLE statement drops all historical records from a system-versioned table.

Historic data is protected from `TRUNCATE` statements, as per the SQL standard, and an Error 4137 is instead raised:

```
TRUNCATE t;
ERROR 4137 (HY000): System-versioned tables do not support TRUNCATE TABLE
```

Excluding Columns From Versioning

Another MariaDB extension allows one to version only a subset of columns in a table. This is useful, for example, if you have a table with user information that should be versioned, but one column is, let's say, a login counter that is incremented often and is not interesting to version. Such a column can be excluded from versioning by declaring it `WITHOUT VERSIONING`

```
CREATE TABLE t (
   x INT,
```

```
   y INT WITHOUT SYSTEM VERSIONING
) WITH SYSTEM VERSIONING;
```

A column can also be declared `WITH VERSIONING`, that will automatically make the table versioned. The statement below is equivalent to the one above:

```
CREATE TABLE t (
   x INT WITH SYSTEM VERSIONING,
   y INT
);
```

System Variables

A number of system variables are related to system-versioned tables:

system_versioning_alter_history

Description: SQL:2011 does not allow ALTER TABLE on system-versioned tables. When this variable is set to `ERROR`, an attempt to alter a system-versioned table will result in an error. When this variable is set to `KEEP`, `ALTER TABLE` will be allowed, but the history will become incorrect — querying historical data will show the new table structure. This mode is still useful, for example, when adding new columns to a table. Note that if historical data contains or would contain nulls, attempting to `ALTER` these columns to be `NOT NULL` will return an error (or warning if strict_mode is not set).

Command line: `--system-versioning-alter-history=value`

Valid Values: `ERROR`, `KEEP`

system_versioning_asof

Description: If set to a specific timestamp value, an implicit `FOR SYSTEM_TIME AS OF` clause will be applied to all queries. This is useful if one wants to do many queries for history at the specific point in time. Set it to `'DEFAULT'` to restore the default behavior. Has no effect on DML, so queries such as INSERT .. SELECT and REPLACE .. SELECT need to state AS OF explicitly.

Note: You need to use quotes around the name `'DEFAULT'` when setting the session value, unquoted literal `DEFAULT` will restore the current global value instead.

system_versioning_innodb_algorithm_simple

Description: Never fully implemented and removed in the following release.

Command line: `--system-versioning-innodb-algorithm-simple[={0|1}]`

Introduced: MariaDB 10.3.4

system_versioning_insert_history

Description: Allows direct inserts into ROW_START and ROW_END columns if secure_timestamp allows changing timestamp.

Command line: `--system-versioning-insert-history[={0|1}]`

Introduced: MariaDB 10.11.0

Limitations

Versioning clauses cannot be applied to generated (virtual and persistent) columns.

mariadb-dump did not read historical rows from versioned tables, and so

historical data would not be backed up. Also, a restore of the timestamps would not be possible as they cannot be defined by an insert/a user. From MariaDB 10.11, use the `-H` or `--dump-history` options to include the history.

Transaction-precise system-versioned tables do not support partitioning by `SYSTEM_TIME`. See Transaction-Precise History in InnoDB for details.

See Also

mysql.transaction_registry Table

MariaDB Temporal Tables (video)

This page is licensed: CC BY-SA / Gnu FDL